



slinkinone

14 сен 2015 в 15:33

PE (Portable Executable): На странных берегах

🕒 18 мин 👁 181K

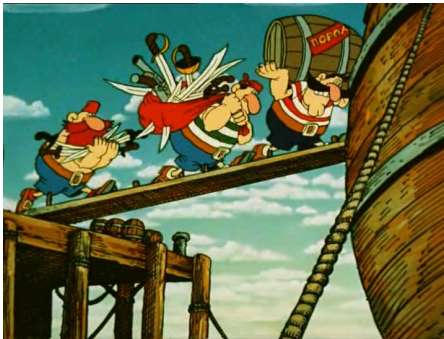
Assembler*, Windows*

[Из песочницы](#)

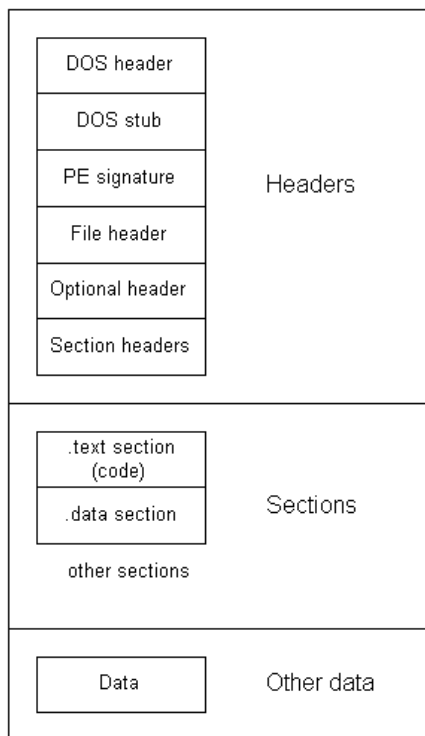
Эта статья представляет из себя рассказ о том как устроены исполняемые файлы (в точку! Это именно те штуки, которые получаются после компиляции приложений с расширением .exe). После того, как написан код, подключены библиотеки, подгружены к проекту ресурсы (иконки для окон, какие-либо текстовые файлы, картинки и прочее) всё это компонуется в один единственный исполняемый файл, преимущественно с расширением .exe. Вот именно в этот омут мы и погрузимся.

**Статья находится под эгидой «для начинающих» поэтому будет изобиловать схемами и описанием важных элементов загрузки.*

Введение



PE формат — это формат исполняемых файлов всех 32- и 64- разрядных Windows систем. На данный момент существует два формата PE-файлов: PE32 и PE32+. PE32 формат для x86 систем, а PE32+ для x64. Описанные структуры можно наблюдать в заголовочном файле [WINNT.h](#), который поставляется вместе SDK. Описание сего формата от microsoft можно скачать [здесь](#), а я же пока оставлю здесь небольшое схематическое представление. Просто пробежитесь глазами, в процессе статьи вы начнёте схватывать и всё разложится по полочкам.



Любой файл, это есть лишь последовательность байт. А формат, это как специальная карта (сокровищ) для него. То есть показывает что где находится, где острова с кокосами, где с бананами, где песчаные берега, а где Сомалийские, куда лучше-бы не соваться. Так давайте же изучим широкие просторы этого океана. Отдать швартовы!

«Сейчас вы услышите грустную истори. о мальчике Бобби»
(Остров сокровищ)

Dos-Header (IMAGE_DOS_HEADER) и Dos-stub



Dos заголовок. Это самая первая структура (самый первый островок который нам встретился на пути) в файле и она имеет размер 64 байта. В этой структуре наиболее важные поля это *e_magic* и *e_lfnew*. Посмотрим как выглядит структура:

DOS MZ Header

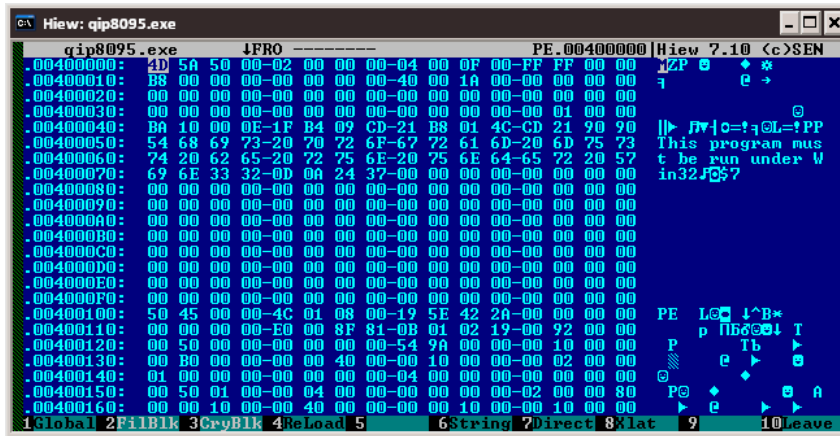
```
typedef struct _IMAGE_DOS_HEADER {
    0x00 WORD e_magic;
    0x02 WORD e_cblp;
    0x04 WORD e_cp;
    0x06 WORD e_crlc;
    0x08 WORD e_cparhdr;
    0x0a WORD e_minalloc;
    0x0c WORD e_maxalloc;
    0x0e WORD e_ss;
    0x10 WORD e_sp;
    0x12 WORD e_csum;
    0x14 WORD e_ip;
    0x16 WORD e_cs;
    0x18 WORD e_lfarlc;
    0x1a WORD e_ovno;
    0x1c WORD e_res[4];
    0x24 WORD e_oemid;
    0x26 WORD e_oeminfo;
    0x28 WORD e_res2[10];
    0x3c DWORD e_lfanew;
} IMAGE_DOS_HEADER, *PIMAGE_DOS_HEADER;
```

Изучать все поля на данном этапе ни к чему, т.к. особой смысловой нагрузки они не несут. Рассмотрим только те, которые необходимы для загрузки и представляют особенный интерес. (Дальше и ниже по тексту, формат описания полей будет вида *name*: TYPE — description).

e_magic: WORD — сигнатура находящаяся по смещению 0 от начала файла и равная “MZ”. Поговаривают, что MZ сокращение от Марк Збиновски — ~~самый злобный пират на всём водном пространстве~~ ведущий разработчик MS DOS и EXE формата. Если данная сигнатура не равна MZ, то файл не загрузится.

e_lfanew: DWORD — смещение PE заголовка относительно начала файла. PE заголовок должен начинаться с сигнатуры (характерная запись/подпись) PE\0\0. PE заголовок может располагаться в любом месте файла. Если посмотреть на структуру, то можно увидеть, что *e_lfanew* находится по смещению 0x3C (60 в десятичной). То есть чтобы прочитать это значение, мы должны от указателя на начало файла (введём обозначение — *ptrFile*) “плюсовать” 60 байт и тогда мы встанем face to face перед *e_lfanew*. Читаем это значение (пусть будет *peStep*) и плюсуем к *ptrFile* значение *peStep*. Mission completed — мы на месте шеф, это должен быть PE заголовок. А узнать это наверняка мы можем сверив первые четыре байта этого заголовка. Как было сказано выше, они должны равняться PE\0\0.

После 64 первых байт файла стартует dos-stub (пираты также называют его dos заглушка). Эта область в памяти которая в большинстве своём забита нулями. (Взгляните ещё раз на структуру — заглушка лежит после dos-header(a) и перед PE заголовком) Служит она только для обратной совместимости, нынешним системам она ни к чему. В неё может быть записана мини версия dos программы ограниченную в 192 байта (256 — конец заглушки, 64 — размер dos заголовка). Но легче найти Access Point в Зимбабве, нежели такую программу. Стандартное поведение, если запустить программу на dos, то она выведет сообщения вида “This program cannot be run in DOS mode.” или “This program must be run under win32”. Если увидите эти строки, это значит что вы попали... в далёкий 85-ый.



“К чёрту деньги, я говорю о бумагах Флинта!”

(Остров сокровищ)

PE-Header (IMAGE_NT_HEADERS)



Прочитали `e_lfnew`, отступили от начала файла на `peStep` байт. Теперь мы можем начинать анализировать PE заголовок. Это новый для нас остров и он должен располагаться на просторах следующих 0x18 байт. Структура представлена ниже:

```
typedef struct _IMAGE_NT_HEADERS {
    DWORD Signature;
    IMAGE_FILE_HEADER FileHeader;
    IMAGE_OPTIONAL_HEADER OptionalHeader;
} IMAGE_NT_HEADERS, *PIMAGE_NT_HEADERS;
```

[Объяснить с](#)

Это интересная структура, т.к. она содержит в себе подструктуры. Если представить PE файл как океан, каждая структура это материк (или остров). На материках расположены государства, которые могут рассказать о своей территории. А рассказ складывается из истории отдельных городов (полей) в этом государстве. Так вот — NT Header — это материк, который содержит такие страны, как Signature (город-государство), FileHeader, OptionalHeader. Как уже было сказано, *Signature*: DWORD — содержит 4-ёх байтовую сигнатуру, характеризующую формат файла. Рассмотрим что ещё может поведать нам этот материк.

File-Header (IMAGE_FILE_HEADER)

Это страна где вечно стреляют, торгуют наркотиками и занимаются проституцией где каждый город рассказывает в каком идеальном государстве он расположен. Это что касается неформального описания, а формальное заключается в следующем — набор полей, описывающий базовые

характеристики файла. Давайте рассмотрим данную ~~державу~~ структуру:

```
typedef struct _IMAGE_FILE_HEADER {
    WORD Machine;
    WORD NumberOfSections;
    DWORD TimeDateStamp;
    DWORD PointerToSymbolTable;
    DWORD NumberOfSymbols;
    WORD SizeOfOptionalHeader;
    WORD Characteristics;
} IMAGE_FILE_HEADER, *PIMAGE_FILE_HEADER;
```

Объяснить с 

Я лишь сухо опишу данные поля, т.к. названия интуитивно понятные и представляют из себя непосредственные значения, а не VA, RVA, RAW и прочие ~~страшные~~ интригующие штуки, о которых пока, мы только слышали от старых пиратов. Хотя с RAW мы уже сталкивались — это как раз смещения относительно начала файла (их ещё называют сырыми указателями или file offset). То есть если мы имеем RAW адрес, это значит что нужно шагнуть от начала файла на RAW позиций (`ptrFile + RAW`). После можно начинать читать значения. Ярким примером данного вида является `e_lfnew` — что мы рассмотрели выше в Dos заголовке.

**Machine*: WORD — это число (2 байта) задаёт архитектуру процессора, на которой данное приложение может выполняться.

NumberOfSections: DWORD — количество секций в файле. Секции (в дальнейшем будем называть таблицей секций) следуют сразу после заголовка (PE-Header). В документации сказано что количество секций ограничено числом 96.

TimeDateStamp: DWORD — число хранящее дату и время создания файла.

PointerToSymbolTable: DWORD — смещение (RAW) до таблицы символов, а *SizeOfOptionalHeader* — это размер данной таблицы. Данная таблица призвана служить для хранения отладочной информации, но отряд не заметил потери бойца с самого начала службы. Чаще всего это поле зачищается нулями.

SizeOfOptionHeader: WORD — размер опционального заголовка (что следует сразу за текущим) В документации указано, что для объектного файла он устанавливается в 0...

**Characteristics*: WORD — характеристики файла.

* — поля, которые определены диапазоном значений. Таблицы возможных значений представлены в описании структуры на оф. сайте и приводятся здесь не будут, т.к. ничего особо важного для понимая формата они не несут.

Оставим этот остров! Нам нужно двигаться дальше. Ориентир — страна под названием Optional-Header.

“— Где карта, Билли? Мне нужна карта.”
(Остров сокровищ)

Optional-Header (IMAGE_OPTIONAL_HEADER)



Название ~~его материка~~ заголовка не очень удачное. Этот заголовок является обязательным и имеет 2 формата PE32 и PE32+ (IMAGE_OPTIONAL_HEADER32 и IMAGE_OPTIONAL_HEADER64 соответственно). Формат хранится в поле *Magic*: WORD. Заголовок содержит необходимую информацию для загрузки файла. [Как всегда](#):

▼ IMAGE_OPTIONAL_HEADER

```
typedef struct _IMAGE_OPTIONAL_HEADER {
    WORD                Magic;
    BYTE                MajorLinkerVersion;
    BYTE                MinorLinkerVersion;
    DWORD               SizeOfCode;
    DWORD               SizeOfInitializedData;
    DWORD               SizeOfUninitializedData;
    DWORD               AddressOfEntryPoint;
    DWORD               BaseOfCode;
    DWORD               BaseOfData;
    DWORD               ImageBase;
    DWORD               SectionAlignment;
    DWORD               FileAlignment;
    WORD                MajorOperatingSystemVersion;
    WORD                MinorOperatingSystemVersion;
    WORD                MajorImageVersion;
    WORD                MinorImageVersion;
    WORD                MajorSubsystemVersion;
    WORD                MinorSubsystemVersion;
    DWORD               Win32VersionValue;
    DWORD               SizeOfImage;
    DWORD               SizeOfHeaders;
    DWORD               CheckSum;
    WORD                Subsystem;
    WORD                DllCharacteristics;
    DWORD               SizeOfStackReserve;
    DWORD               SizeOfStackCommit;
    DWORD               SizeOfHeapReserve;
    DWORD               SizeOfHeapCommit;
    DWORD               LoaderFlags;
    DWORD               NumberOfRvaAndSizes;
    IMAGE_DATA_DIRECTORY DataDirectory[ IMAGE_NUMBEROF_DIRECTORY_ENTRIES ];
} IMAGE_OPTIONAL_HEADER;
R, *PIMAGE_OPTIONAL_HEADER;
```

Объяснить с

* Как всегда, мы изучим только основные поля, которые имеют наибольшее влияние на представление о загрузке и того, как двигаться дальше по файлу. Давайте условимся — в полях

данной структуры, содержатся значения с VA (Virtual address) и RVA (Relative virtual address) адресами. Это уже адреса не такие как RAW, и их нужно уметь читать (точнее считать). Мы непременно научимся это делать, но только для начала разберём структуры, которые идут друг за другом, чтобы не запутаться. Пока просто запомните — это адреса, которые после расчётов, указывают на определённое место в файле. Также встретится новое понятие — выравнивание. Его мы рассмотрим в купе с RVA адресами, т.к. эти они довольно тесно связаны.

AddressOfEntryPoint: DWORD — RVA адрес точки входа. Может указывать в любую точку адресного пространства. Для .exe файлов точка входа соответствует адресу, с которого программа начинает выполняться и не может равняться нулю!

BaseOfCode: DWORD — RVA начала кода программы (секции кода).

BaseOfData: DWORD — RVA начала кода программы (секции данных).

ImageBase: DWORD — предпочтительный базовый адрес загрузки программы. Должен быть кратен 64кб. В большинстве случаев равен 0x00400000.

SectionAlign: DWORD — размер выравнивания (байты) секции при выгрузке в виртуальную память.

FileAlign: DWORD — размер выравнивания (байты) секции внутри файла.

SizeOfImage: DWORD — размер файла (в байтах) в памяти, включая все заголовки. Должен быть кратен *SectionAlign*.

SizeOfHeaders: DWORD — размер всех заголовков (DOS, DOS-Stub, PE, Section) выравненный на *FileAlign*.

NumberOfRvaAndSizes: DWORD — количество каталогов в таблице директорий (ниже сама таблица). На данный момент это поле всегда равно символической константе `IMAGE_NUMBEROF_DIRECTORY_ENTRIES`, которая равна 16-ти.

DataDirectory[*NumberOfRvaAndSizes*]: `IMAGE_DATA_DIRECTORY` — каталог данных. Проще говоря это массив (размером 16), каждый элемент которого содержит структуру из 2-ух DWORD-ых значений.

Рассмотрим что из себя представляет структура `IMAGE_DATA_DIRECTORY`:

```
typedef struct _IMAGE_DATA_DIRECTORY {
    DWORD VirtualAddress;
    DWORD Size;
} IMAGE_DATA_DIRECTORY, *PIMAGE_DATA_DIRECTORY;
```

[Объяснить с](#) 

Что мы имеем? Мы имеем массив из 16 элементов, каждый элемент которого, содержит адрес и размер (чего? как? зачем? всё через минуту). Встаёт вопрос чего именно это характеристики. Для этого, у microsoft имеются специальные константы для соответствия. Их можно увидеть в самом конце описания структуры. А пока:

```
// Directory Entries
#define IMAGE_DIRECTORY_ENTRY_EXPORT      0 // Export Directory
#define IMAGE_DIRECTORY_ENTRY_IMPORT      1 // Import Directory
#define IMAGE_DIRECTORY_ENTRY_RESOURCE    2 // Resource Directory
#define IMAGE_DIRECTORY_ENTRY_EXCEPTION   3 // Exception Directory
#define IMAGE_DIRECTORY_ENTRY_SECURITY    4 // Security Directory
#define IMAGE_DIRECTORY_ENTRY_BASERELOC    5 // Base Relocation Table
#define IMAGE_DIRECTORY_ENTRY_DEBUG        6 // Debug Directory
//      IMAGE_DIRECTORY_ENTRY_COPYRIGHT    7 // (X86 usage)
#define IMAGE_DIRECTORY_ENTRY_ARCHITECTURE 7 // Architecture Specific Data
#define IMAGE_DIRECTORY_ENTRY_GLOBALPTR    8 // RVA of GP
#define IMAGE_DIRECTORY_ENTRY_TLS          9 // TLS Directory
```



```
#define IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG    10 // Load Configuration Directory
#define IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT    11 // Bound Import Directory in headers
#define IMAGE_DIRECTORY_ENTRY_IAT            12 // Import Address Table
#define IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT    13 // Delay Load Import Descriptors
#define IMAGE_DIRECTORY_ENTRY_COM_DESCRIPTOR 14 // COM Runtime descriptor
```

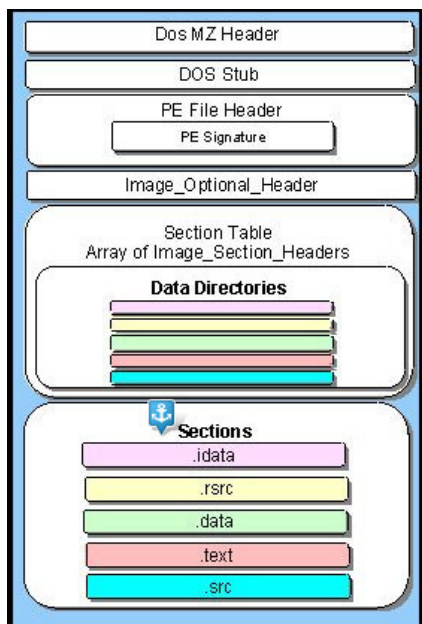
[Объяснить с](#)

Ага! Мы видим, что каждый элемент массива, отвечает за прикрепленную к нему таблицу. Но увы и ах, пока эти берега недостижимы для нас, т.к. мы не умеем работать с VA и RVA адресами. А для того чтобы научиться, нам нужно изучить что такое секции. Именно они расскажут о своей структуре и работе, после чего станет понятно для чего нужны VA, RVA и выравнивания. В рамках данной статьи, мы затронем только экспорт и импорт. Предназначение остальных полей можно найти в оф. документации, либо в книжках. Так вот. Собственно поля:

VirtualAddress: DWORD — RVA на таблицу, которой соответствует элемент массива.

Size: DWORD — размер таблицы в байтах.

Итак! Чтобы добраться до таких экзотических берегов как таблицы импорта, экспорта, ресурсов и прочих, нам необходимо пройти квест с секциями. Ну что ж юнга, взглянем на общую карту, определим где мы сейчас находимся и будем двигаться дальше:



А находимся мы не посредственно перед широкими просторами секций. Нам нужно непременно выпытать что они таят и разобраться уже наконец с другим видом адресации. Нам хочется настоящих приключений! Мы хотим поскорее отправится к таким республикам как таблицы импорта и экспорта. Старые пираты говаривают, что не каждый смог до них добраться, а тот кто добрался ~~вернулся с золотом и женщинами~~ со священными знаниями об океане. Отчаливаем и держим путь на Section header.

“- Ты низложен, Сильвер! Слезай с бочки!”
(Остров сокровищ)

Section-header (IMAGE_SECTION_HEADER)



Сразу за массивом *DataDirectory* друг за другом идут секции. Таблица секций представляет из себя суверенное государство, которое делится на *NumberOfSections* городов. Каждый город имеет своё ремесло, свои права, а также размер в 0x28 байт. Количество секций указано в поле *NumberOfSections*, что хранится в File-header-е. Итак, рассмотрим [структуру](#):

```
typedef struct _IMAGE_SECTION_HEADER {
    BYTE  Name[IMAGE_SIZEOF_SHORT_NAME];
    union {
        DWORD PhysicalAddress;
        DWORD VirtualSize;
    } Misc;
    DWORD VirtualAddress;
    DWORD SizeOfRawData;
    DWORD PointerToRawData;
    DWORD PointerToRelocations;
    DWORD PointerToLinenumbers;
    WORD  NumberOfRelocations;
    WORD  NumberOfLinenumbers;
    DWORD Characteristics;
} IMAGE_SECTION_HEADER, *PIMAGE_SECTION_HEADER;
```

[Объяснить с](#)

Name: BYTE[IMAGE_SIZEOF_SHORT_NAME] — название секции. На данный момент имеет длину в 8 символов.

VirtualSize: DWORD — размер секции в виртуальной памяти.

SizeOfRawData: DWORD — размер секции в файле.

VirtualAddress: DWORD — RVA адрес секции.

SizeOfRawData: DWORD — размер секции в файле. Должен быть кратен *FileAlignment*.

PointerToRawData: DWORD — RAW смещение до начала секции. Также должен быть кратен *FileAlignment*...

Characteristics: DWORD — атрибуты доступа к секции и правила для её загрузки в вирт. память. Например атрибут для определения содержимого секции (иниц. данные, не инициал. данные, код). Или атрибуты доступа — чтение, запись, исполнение. Это не весь их спектр. Характеристики задаются константами из того-же WINNT.h, которые начинаются с IMAGE_SCN_. Более подробно ознакомится с атрибутами секций можно [здесь](#). Также хорошо описаны атрибуты в книгах Криса Касперски — список литературы в конце статьи.

По поводу имени следует запомнить следующее — секция с ресурсам, всегда должна иметь имя .rsrc. В противном случае ресурсы не будут подгружены. Что касается остальных секций — то имя может быть любым. Обычно встречаются осмысленные имена, например .data, .src и т.д... Но бывает и такое:

```

File Edit View Search Terminal Help
0008b060 00000000 00000000 00000000 00000000 00000000
Sections:
Idx Name      Size      VMA      LMA      File off  Algn
 0 NOTE      00028a01 00401000 00401000 00000400 2**2
 1 FOR       00000e00 00475000 00475000 00028e00 2**2
 2 AGAiN     00000000 00477000 00477000 00029c00 2**2
 3 GUYS:     00000c00 0047a000 0047a000 00029c00 2**2
 4 PLEASE    00000000 0047d000 0047d000 0002a800 2**2
 5 LEAVE     00000200 0047e000 0047e000 0002a800 2**2
 6 MY SRC    00000000 0047f000 0047f000 0002aa00 2**2
 7 AND IDEA  00002000 00486000 00486000 0002aa00 2**2
 8 ALONE,    00001600 0048a000 0048a000 0002ca00 2**2
 9 THANX!    00000000 0048c000 0048c000 0002e000 2**2

```

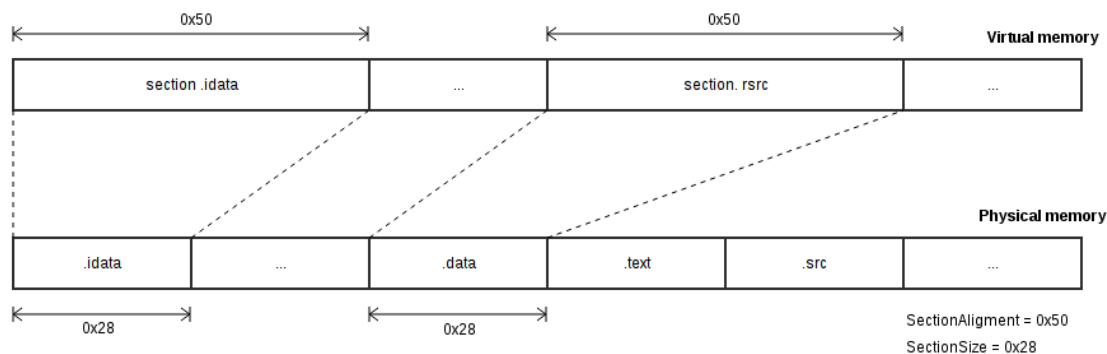
Секции, это такая область, которая выгружается в виртуальную память и вся работа происходит непосредственно с этими данными. Адрес в виртуальной памяти, без всяких смещений называется Virtual address, сокращённо VA. Предпочитаемый адрес для загрузки приложения, задаётся в поле *ImageBase*. Это как точка, с которой начинается область приложения в виртуальной памяти. И относительно этой точки отсчитываются смещения RVA (Relative virtual address). То есть $VA = ImageBase + RVA$; *ImageBase* нам всегда известно и получив в своё распоряжение VA или RVA, мы можем выразить одно через другое.

Тут вроде освоились. Но это же виртуальная память! А мы то находимся в физической. Виртуальная память для нас сейчас это как путешествие в другие галактики, которые мы пока можем лишь только представлять. Так что в виртуальную память нам на данный момент не попасть, но мы можем узнать что там будет, ведь это взято из нашего файла.

Выравнивание



Для того чтобы правильно представлять выгрузку в вирт. память, необходимо разобраться с таким механизмом как выравнивание. Для начала давайте взглянем на схему того, как секции выгружаются в память.



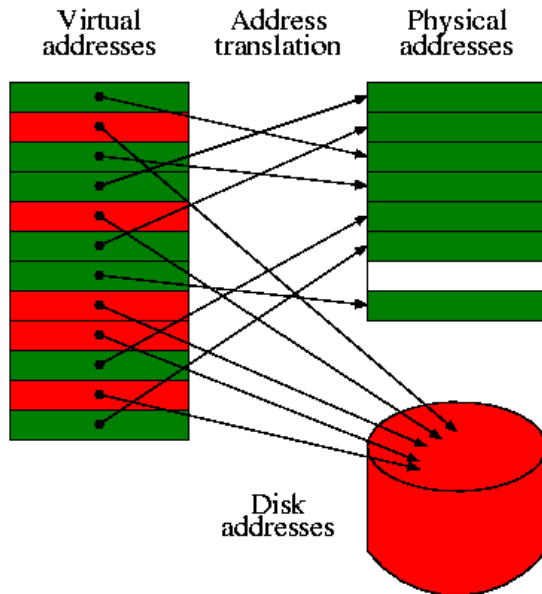
Как можно заметить, секция выгружается в память не по своему размеру. Здесь используются выравнивания. Это значение, которому должны быть кратен размер секции в памяти. Если посмотреть на схему, то мы увидим, что размер секции 0x28, а выгружается в размере 0x50. Это происходит из-за размера выравнивания. 0x28 “не дотягивает” до 0x50 и как следствие, будет выгружена секция, а остальное пространство в размере 0x50-0x28 занулится. А если размер секции был бы больше размера выравнивания, то что? Например `sectionSize = 0x78`, а `sectionAlign = 0x50`, т.е. остался без изменений. В таком случае, секция занимала бы в памяти 0xA0 (0xA0 = 0x28 * 0x04) байт. То есть значение которое кратно `sectionAlign` и полностью кроет `sectionSize`. Следует отметить, что секции в файле выравниваются аналогичным образом, только на размер `FileAlign`. Получив необходимую базу, мы можем разобраться с тем, как конвертировать из RVA в RAW.

“Здесь вам не равнина, здесь климат иной.”
(В.С. Высоцкий)

Небольшой урок арифметики



Перед тем как начать выполнение, какая то часть программы должна быть отправлена в адресное пространство процессора. Адресное пространство — это объём физически адресуемой процессором оперативной памяти. “Кусок” в адресном пространстве, куда выгружается программа называется виртуальным образом (virtual image). Образ характеризуется адресом базовой загрузки (Image base) и размером (Image size). Так вот VA (Virtual address) — это адрес относительно начала виртуальной памяти, а RVA (Relative Virtual Address) относительно места, куда была выгружена программа. Как узнать базовый адрес загрузки приложения? Для этого существует отдельное поле в опциональном заголовке под названием *ImageBase*. Это была небольшая прелюдия чтобы освежить в памяти. Теперь рассмотрим схематичное представление разных адресаций:



Дак как же всё таки прочитать информацию из файла, не выгружая его в виртуальную память? Для этого нужно конвертировать адреса в RAW формат. Тогда мы сможем внутри файла шагнуть на нужный нам участок и прочитать необходимые данные. Так как RVA — это адрес в виртуальной памяти, данные по которому были спроецированы из файла, то мы можем произвести обратный процесс. Для этого нам понадобится ~~ключ-девять-на-шестнадцать~~ простая арифметика. Вот несколько формул:

```
VA = ImageBase + RVA;
RAW = RVA - sectionRVA + rawSection;
// rawSection - смещение до секции от начала файла
// sectionRVA - RVA секции (это поле хранится внутри секции)
```

[Объяснить с 🗨️](#)

Как видно, чтобы высчитать RAW, нам нужно определить секцию, которой принадлежит RVA. Для этого нужно пройти по всем секциям и проверить следующие условие:

```
RVA >= sectionVirtualAddress && RVA < ALIGN_UP(sectionVirtualSize, sectionAlign)
// sectionAlign - выравнивание для секции. Значение можно узнать в Optional-header.
// sectionVirtualAddress - RVA секции - хранится непосредственно в секции
// ALIGN_UP() - функция, определяющая сколько занимает секция в памяти, учитывая выравнивание
```

[Объяснить с 🗨️](#)

Сложив все пазлы, получим вот такой листинг:

```
typedef uint32_t DWORD;
typedef uint16_t WORD;
typedef uint8_t BYTE;

#define ALIGN_DOWN(x, align) (x & ~(align-1))
#define ALIGN_UP(x, align) ((x & (align-1)) ? ALIGN_DOWN(x, align) + align : x)

// IMAGE_SECTION_HEADER sections[numbersOfSections];
```

```
// init array sections

int defSection(DWORD rva)
{
    for (int i = 0; i < numberOfSection; ++i)
    {
        DWORD start = sections[i].VirtualAddress;
        DWORD end = start + ALIGN_UP(sections[i].VirtualSize, sectionAlignment);

        if(rva >= start && rva < end)
            return i;
    }
    return -1;
}

DWORD rvaToOff(DWORD rva)
{
    int indexSection = defSection(rva);
    if(indexSection != -1)
        return rva - sections[indexSection].VirtualAddress + sections[indexSection].PointerToRawData;
    else
        return 0;
}
```

[Объяснить с](#)

*Я не стал включать в код объявление типа, и инициализацию массива, а лишь предоставил функции, которые помогут при расчёте адресов. Как видите, код получился не очень сложным. Разве что малость запутанным. Это проходит... если уделить ещё немного времени колупанию в .exe через дизассемблер.

УРА! Разобрались. Теперь мы можем отправиться в края ресурсов, библиотек импорта и экспорта и вообще куда душа желает. Мы ведь только что научились работать с новым видом адресации. В путь!



“Неплохо, неплохо! Всё же они получили свой паёк на сегодня!”
(Остров сокровищ)

Export table



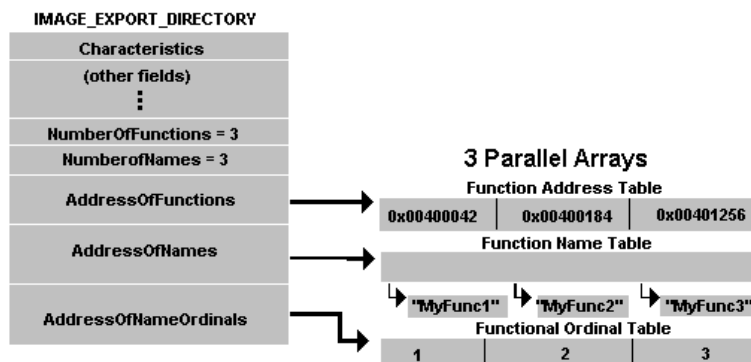
В самом первом элементе массива *DataDirectory* хранится RVA на таблицу экспорта, которая представлена структурой *IMAGE_EXPORT_DIRECTORY*. Эта таблица свойственна файлам динамических библиотек (.dll). Основной задачей таблицы является связь экспортируемых функций

с их RVA. Описание представлено в [оф. спецификации](#):

```
typedef struct _IMAGE_EXPORT_DIRECTORY {
    DWORD Characteristics;
    DWORD TimeDateStamp;
    WORD MajorVersion;
    WORD MinorVersion;
    DWORD Name;
    DWORD Base;
    DWORD NumberOfFunctions;
    DWORD NumberOfNames;
    DWORD AddressOfFunctions;
    DWORD AddressOfNames;
    DWORD AddressOfNameOrdinals;
} IMAGE_EXPORT_DIRECTORY, *PIMAGE_EXPORT_DIRECTORY;
```

[Объяснить с](#) 

Эта структура содержит три указателя на три разные таблицы. Это таблица имён (функций) (*AddressOfNames*), ординалов (*AddressOfNameOrdinals*), адресов (*AddressOfFunctions*). В поле *Name* хранится RVA имени динамической библиотеки. Ординал — это как посредник, между таблицей имён и таблицей адресов, и представляет из себя массив индексов (размер индекса равен 2 байта). Для большей наглядности рассмотрим схему:



Рассмотрим пример. Допустим *i*-ый элемент массива имён указывает на название функции. Тогда адрес этой функции можно получить обратившись к *i*-му элементу в массиве адресов. Т.е. *i* — это ординал.

Внимание! Если вы взяли к примеру 2-ой элемент в таблице ординалов, это не значит 2 — это ординал для таблиц имён и адресов. Индексом является значение, хранящееся во втором элементе массива ординалов.

Количество значений в таблицах имён (*NumberOfNames*) и ординалов равны и не всегда совпадают с количеством элементов в таблице адресов (*NumberOfFunctions*).

“За мной пришли. Спасибо за внимание. Сейчас, должно быть, будут убивать!”
(Остров сокровищ)

Import table

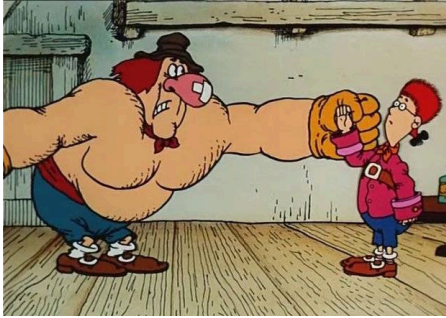


Таблица импорта неотъемлемая часть любого приложения, которая использует динамические библиотеки. Данная таблица помогает соотнести вызовы функций динамических библиотек с соответствующими адресами. Импорт может происходить в трёх разных режимах: стандартный, связывающем (bound import) и отложенном (delay import). Т.к. тема импорта достаточно многогранна и тянет на отдельную статью, я опишу только стандартный механизм, а остальные опишу только «скелетом».

Стандартный импорт — в *DataDirectory* под индексом `IMAGE_DIRECTORY_ENTRY_IMPORT(=1)` хранится таблица импорта. Она представляет собой массив из элементов типа `IMAGE_IMPORT_DESCRIPTOR`. Таблица импорта хранит (массивом) имена функций/ординалов и в какое место загрузчик должен записать эффективный адрес этой функций. Этот механизм не очень эффективен, т.к. откровенно говоря всё сводится к перебору всей таблицы экспорта для каждой необходимой функции.

Bound import — при данной схеме работы в поля (в первом элементе стандартной таблицы импорта) `TimeStamp` и `ForwardChain` заносится -1 и информация о связывании хранится в ячейке *DataDirectory* с индексом `IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT(=11)`. То есть это своего рода флаг загрузчику о том что нужно использовать bound import. Так же для «цепочки bound импорта» фигурируют свои структуры. Алгоритм работы заключается в следующем — в виртуальную память приложения выгружается необходимая библиотека и все необходимые адреса «биндятся» ещё на этапе компиляции. Из недостатков можно отметить то, что при перекомпиляции dll, нужно будет перекомпилировать само приложение, т.к. адреса функций будут изменены.

Delay import — при данном методе подразумевается что .dll файл прикреплён к исполняемому, но в память выгружается не сразу (как в предыдущих двух методах), а только при первом обращении приложения к символу (так называют выгружаемые элементы из динамических библиотек). То есть программа выполняется в памяти и как только процесс дошёл до вызова функции из динамической библиотеки, то вызывается специальный обработчик, который подгружает dll и разносит эффективные адреса её функций. За отложенным импортом загрузчик обращается к `DataDirectory[IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT]` (элемент с номером 15).

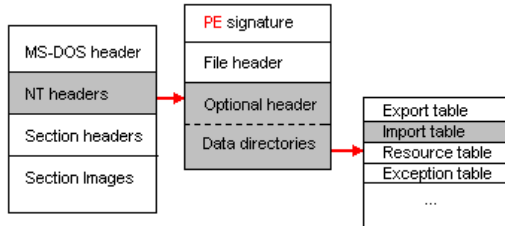
Мало осветив методы импорта, перейдём непосредственно к таблице импорта.

“-Это моряк! Одежда у него была морская. — Да ну? А ты думал найти здесь епископа?”
(Остров сокровищ — Джон Сильвер)

Import-descriptor (`IMAGE_IMPORT_DESCRIPTOR`)



Для того чтобы узнать координаты таблицы импорта, нам нужно обратиться к массиву *DataDirectory*. А именно к элементу *IMAGE_DIRECTORY_ENTRY_IMPORT* (=1). И прочитать RVA адрес таблицы. Вот общая схема пути, который требуется проделать:



Затем из RVA получаем RAW, в соответствии с формулами приведёнными выше, и затем “шагаем” по файлу. Теперь мы впритык перед массивом структур под названием *IMAGE_IMPORT_DESCRIPTOR*. Признаком конца массива служит “нулевая” структура.

```
typedef struct _IMAGE_IMPORT_DESCRIPTOR {
    union {
        DWORD Characteristics;
        DWORD OriginalFirstThunk;
    } DUMMYUNIONNAME;
    DWORD TimeDateStamp;
    DWORD ForwarderChain;
    DWORD Name;
    DWORD FirstThunk;
} IMAGE_IMPORT_DESCRIPTOR, *PIMAGE_IMPORT_DESCRIPTOR;
```

Объяснить с

Я не смог выудить на msdn ссылку на описание структуры, но вы можете наблюдать её в файле WINNT.h. Начнём разбираться.

OriginalFirstThunk: DWORD — RVA таблицы имён импорта (INT).

TimeDateStamp: DWORD — дата и время.

ForwarderChain: DWORD — индекс первого переправленного символа.

Name: DWORD — RVA строки с именем библиотеки.

FirstThunk: DWORD — RVA таблицы адресов импорта (IAT).

Тут всё несколько похоже на экспорт. Также таблица имён (INT) и ~~и тоже рублище на нём~~ адресов (IAT). Также RVA имени библиотеки. Только вот INT и IAT ссылаются на массив структур *IMAGE_THUNK_DATA*. Она представлена в двух формах — для 64- и для 32-й систем и различаются только размером полей. Рассмотрим на примере x86:

```
typedef struct _IMAGE_THUNK_DATA32 {
    union {
```

```

DWORD ForwarderString;
DWORD Function;
DWORD Ordinal;
DWORD AddressOfData;

} u1;
} IMAGE_THUNK_DATA32, *PIMAGE_THUNK_DATA32;

```

[Объяснить с](#)

Важно ответить, что дальнейшие действия зависят от старшего бита структуры. Если он установлен, то оставшиеся биты представляют из себя номер импортируемого символа (импорт по номеру). В противном случае (старший бит сброшен) оставшиеся биты задают RVA импортируемого символа (импорт по имени). Если мы имеем импорт по имени, то указатель хранит адрес на следующую структуру:

```

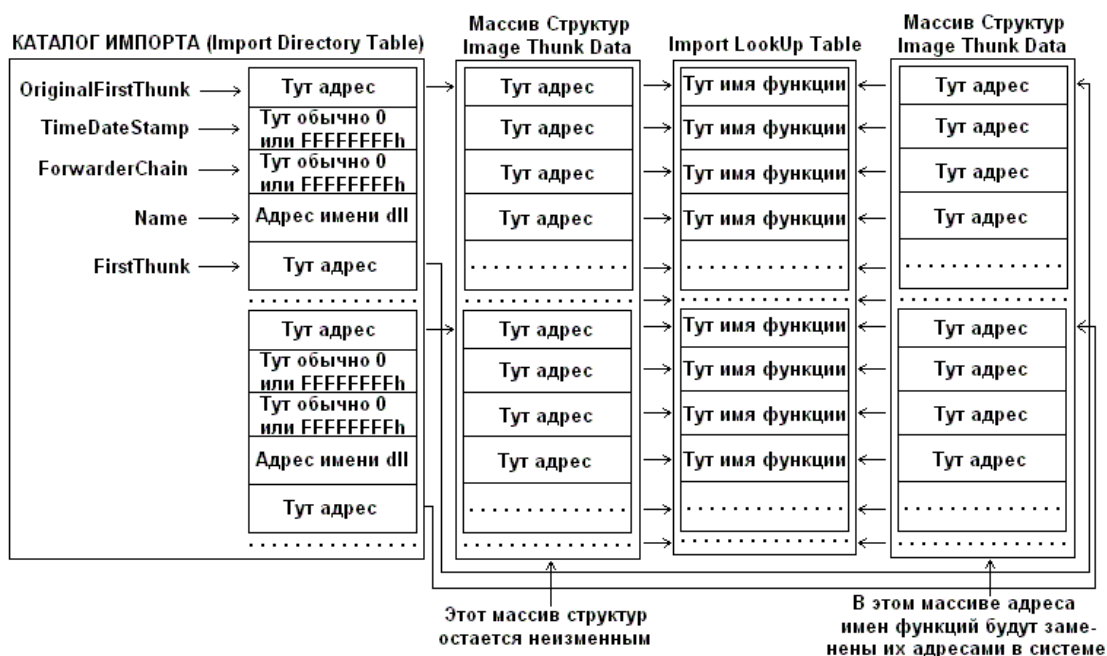
typedef struct _IMAGE_IMPORT_BY_NAME {
    WORD    Hint;
    BYTE    Name[1];
} IMAGE_IMPORT_BY_NAME, *PIMAGE_IMPORT_BY_NAME;

```

[Объяснить с](#)

Здесь *Hint* — это номер функции, а *Name* — имя.

Для чего это всё? Все эти массивы, структуры... Рассмотрим для наглядности замечательную схему с [exelab](#):



Что здесь происходит... Поле *OriginalFirstThunk* ссылается на массив, где хранится информация по импортируемым функциям. Поле *FirstThunk* ссылается на аналогичный массив той же размерности, но разве что заполняется он во время загрузки эффективным адресами функций. Т.е. загрузчик анализирует *OriginalFirstThunk*, определяет реальный адрес функции для каждого его элемента и заносит этот адрес в *FirstThunk*. Другими словами происходит связывание импортируемых символов.

“Мне не нравится эта экспедиция! Мне не нравятся эти матросы! И вообще... что?!!! А, да! Нет! Мне вообще ничего не нравится, сэр!”
(Остров сокровищ — Капитан Смоллетт)

За бортом



В статье была представлена лишь самая база по исполняемым файлам. Не затронуты другие виды импорта, поведение при конфликтующих (например физический размер секции, больше виртуального) или неоднозначных (в том же импорте — вопрос к какому методу прибегнуть) ситуациях. Но это всё уже для более детально изучения и зависит от конкретных загрузчиков в ОС и компиляторов, которые собрали программу. Также не затронуты каталоги ресурсов, отладки и прочих. Тем кому стало интересно, можно почитать более подробные руководства представленные в списке литературы в конце статьи.

“Скажи, Окорок, долго мы будем влиять, как маркитантская лодка? Мне до смерти надоел капитан. Хватит ему командовать! Я хочу жить в его каюте.”
(Остров сокровищ)

Заключение



После того как мы вернулись из путешествия, немного подытожу, что мы увидели и что вынесли. Сегодня мы многое поняли. А именно опишу процесс загрузки приложения в общих словах.

- Сначала происходит считывание заголовков и проверка их на то, что файл является исполняемым. В противном случае работа прекращается, не успев начаться.
- Загрузчик выделяет под приложение требуемый объём виртуальной памяти. Если это возможно, то приложение будет загружено по предпочтительному адресу. Если же нет, то под приложение выделится другой участок памяти и загрузится с этого адреса.
- Затем для каждой секции вычисляется её адрес в виртуальной памяти (относительно базового адреса загрузки) и требуемый размер. После чего для данной области устанавливаются

атрибуты и секция выгружается в память.

- Если базовый адрес отличается от предпочтительного, то происходит настройка адресов.
- Выполняется анализ таблицы импорта и подтягивются необходимые dll. Затем происходит процесс связывания.

На это статья заканчивается. Я думаю этой информации вполне достаточно чтобы иметь базовое представление об исполняемых файлах. Самым любознательным путем на [exelab](#), [wasm](#), [msdn](#), [ассемблеру](#) и дизассемблеру.

Для более чёткого понимания можно порекомендовать поизучать диаграммы. Это действительно даёт более полную картину происходящего внутри. В качестве примера могу предложить вот эту [статью](#) пользователя [@alizar](#) или более общие схемы в [гугле](#). Надеюсь вам понравилось наше путешествие.

Список литературы

ru.wikipedia.org/wiki/Portable_Executable

msdn.microsoft.com/en-us/library/ms809762.aspx

acmvm2.srv.mst.edu/wp-content/uploads/2014/07/PE-Header-Bible.pdf

cs.usu.edu.ru/docs/pe

Крис Касперски — Техника отладки программ без исходных текстов
exelab.ru/faq

Обновления

- **Win32VersionValue** для 32-разрядных ОС должен быть равен 0. ([@vitokop](#))

Теги: [assembler](#), [va](#), [rva](#), [raw](#), [pe](#), [windows](#), [exe](#), [dll](#), [Билли](#)

Хабы: [Assembler](#), [Windows](#)

♦ +26

📖 432



💬 22

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



24

0.1

Карма Общий рейтинг

Вячеслав Слинкин [@slinkinone](#)

R&D, Traffic Analysis

Подписаться



[Сайт](#)

Комментарии 22



 **EvilsInterrupt**
14 сен 2015 в 15:40

А чем Ваш мануал лучше того что написано на wasm.ru?



Ответить



 **slinkinone**
14 сен 2015 в 15:47

При подаче материала я попытался объяснить всё более наглядно, посредством абстраций, диаграмм и схем. Мне кажется, для первого знакомства как раз такая манера изложения будет наиболее удачной, т.к. формирует в голове своего рода «картинку», которую легче воспринимать.



Ответить



 **EvilsInterrupt**
14 сен 2015 в 15:54

Я думаю Вам нужно озвучить это. Потому что информации про PE формат более чем достаточно.



Ответить



 **slinkinone**
14 сен 2015 в 16:21

Добавил в описание пометку о характере статьи.



Ответить



 **CodeRush**
14 сен 2015 в 16:27

Если из статьи слить воду и убрать «Сомалийские острова», ее можно сократить примерно втрое. Кому действительно нужно писать свой парсер PE32 — обратите внимание на [открытую реализацию](#) в radare2.



Ответить



 **MacIn**
14 сен 2015 в 18:22

x86 же.



Ответить



 **slinkinone**
14 сен 2015 в 20:43

Спасибо, исправил.



Ответить



 **artem_kovardin**
14 сен 2015 в 18:50

А есть что-то похожее, только для elf?



Ответить



 **slinkinone**
14 сен 2015 в 20:46

Пока нет.



Ответить



- **Ununtrium**
14 сен 2015 в 19:00
- Для не-системщиков материал интересный, но ощущение что написан для детей.
- ↑  ↓ [Ответить](#)  
- **Wedmer**
16 сен 2015 в 06:03
- Через игру большая часть материала лучше усваивается.
- ↑  ↓ [Ответить](#)  
- **Koval97**
23 янв 2022 в 21:22
- Согласен. Тут сам Бог велел писать несколько серьёзных глав, учитывая что читатель определенно подкован. Оправдываться представлением о некоем "среднестатистическом читателе" - чистое лицемерие в данном случае.
- ↑  ↓ [Ответить](#)  
- **bolk**
14 сен 2015 в 19:42
- Если данная сигнатура не равна MZ, то файл не загрузится.
- Во времена DOS работала так же сигнатура ZM. Не знаю, работает ли одна до сих пор.
- ↑  ↓ [Ответить](#)  
- **morgot**
26 янв 2022 в 00:15
- Нет, винда не даст такое запустить. Проверил на XP-10.
- Точно такое было? Может речь о проверке сигнатуры на 'ZM' ? для оптимизации часто делают что-то вида CMP WORD PTR [Тут адрес дос хидера], 'ZM' (т.к. ворды в памяти хранятся перевёрнутыми).
- ↑  ↓ [Ответить](#)  
- **bolk**
26 янв 2022 в 09:36
- Точно, вот, посмотрите: [https://ru.wikipedia.org/wiki/MZ_\(формат\)](https://ru.wikipedia.org/wiki/MZ_(формат))
- ↑  ↓ [Ответить](#)  
- **morgot**
26 янв 2022 в 18:59
- Спасибо, не знал.
- В любом случае, на всей линейке NT (современной) это уже убрали.
- ↑  ↓ [Ответить](#)  
- **Bombus**
14 сен 2015 в 21:13
- Кажется не хватает про Relocation table.
- ↑  ↓ [Ответить](#)  
- **TrueBers**
14 сен 2015 в 23:28

▸ [Вместо тысячи слов...](#)



Ответить



Deamhan
21 сен 2015 в 14:35

На самом деле статья описывает «идеальные» PE, точно соответствующие документации от MS. В «живой природе» обитает множество PE-шников, этой документации соответствующих лишь частично, но при этом работоспособных. Наиболее распространённый пример: `OriginalFirstThunk == NULL` (результат упаковки UPX-ом). Подобные случаи, пожалуй, наиболее интересны.



Ответить



fsmoke
25 июн 2018 в 13:17

Сейчас читаю эту статью (освежить память по PE) — нашел ошибки.

TimeDateStamp: WORD — число хранящее дату и время создания файла.
 PointerToSymbolTable: DWORD — смещение (RAW) до таблицы символов, а SizeOfOptionalHeader — это размер данной таблицы. Данная таблица призвана служить для хранения отладочной информации, но отряд не заметил потери бойца с самого начала службы. Чаще всего это поле зачищается нулями.
 SizeOfOptionHeader: WORD — размер опционального заголовка (что следует сразу за текущим) В документации указано, что для объектного файла он устанавливается в 0...

Очевидно вместо первого вхождения SizeOfOptionalHeader должно быть NumberOfSymbols, а второе вхождение — дык вообще с опечатками: заменить «SizeOfOptionHeader» на SizeOfOptionalHeader.

Это происходит из-за размера выравнивания. 0x28 “не дотягивает” до 0x50 и как следствие, будет выгружена секция, а остальное пространство в размере 0x50-0x28 занулится. А если размер секции был бы больше размера выравнивания, то что? Например `sectionSize = 0x78`, а `sectionAligment = 0x50`, т.е. остался без изменений. В таком случае, секция занимала бы в памяти 0xA0 ($0xA0 = 0x28 * 0x04$) байт.

Какие $0x28 * 0x04$ — очевидно же что $0x50 * 2$ — выравнивание на то и выравнивание, что фиксирован размер блока и если 0x78 не влезает в 0x50 — процедура вычисления будет выглядеть на шпатель примерно $(0x78 / 0x50 + 1) * 0x50$. Откуда здесь всплыло 0x28 из первого примера неясно. Ошибка вводящая в заблуждение по поводу выравнивания имхо(особенно для неокрепших).

Дальше читаю. Если найду ещё — напишу...



Ответить



fsmoke
25 июн 2018 в 14:47

`RVA >= sectionVirtualAddress && RVA < ALIGN_UP(sectionVirtualSize, sectionAligment)`

опечатки



Ответить





Внимание! Если вы взяли к примеру 2-ой элемент в таблице ординалов, это не значит 2 — это ординал для таблиц имён и адресов. Индексом является значение, хранящееся во втором элементе массива ординалов.

Вот это вообще непонятно что — в массиве ординалов хранятся ординалы, а не индексы! И ординал, который лежит в по индексу 2 будет однозначно соответствовать фции, которая лежит по индексу 2 с именем, которое лежит по индексу 2.

Количество значений в таблицах имён (NumberOfNames) и ординалов равны и не всегда совпадают с количеством элементов в таблице адресов (NumberOfFunctions).

Исходя из документации, которую Вы же и указали оно всегда совпадает...

DWORD NumberOfNames

The number of elements in the AddressOfNames array. **This value seems always to be identical to the NumberOfFunctions field, and so is the number of exported functions.**

И вот ещё:

To find all the information about the fourth function, you need to look up the **fourth element in each array.**



Зарегистрируйтесь на [Хабре](#), чтобы оставить комментарий

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ



Как мы продавали компьютеры в 90-х. Часть #04. Колбасный авиатор

6 мин 8.4K

+30

9

15



Прозрачный прокси для всей домашней сети на базе Xray: настройка за один вечер

Средний 11 мин 20K

Тutorial

+22

216

38

**Bright_Translate**

6 часов назад

Как обстоят дела с WebAssembly?

Простой

9 мин

5.9K

Обзор

Перевод

+21

15

10

**Tsuoko**

7 часов назад

S3, дизайн-система и испытательный срок: заметки начинающего в Selectel

Простой

12 мин

4.2K

Кейс

+18

5

0

**OldfagGamer**

8 часов назад

Ностальгические игры: Меч и Магия VIII

Простой

16 мин

5K

Ретроспектива

+18

3

4

**vodolaz338**

8 часов назад

000xpda или как я реверсил электронный дневник и нашел ключи в логах

Средний

7 мин

5.5K

Из песочницы

+17

13

6

**varanio**

5 часов назад

Как сделать программиста счастливее

Простой

3 мин

5K

+12

12

9

**beeline_cloud**

1 час назад

Опять за старое? Доступный или открытый код — вечное противостояние, а также продолжающийся рост многообразия лицензий

Простой

6 мин

3.3K

Аналитика

+11

4

0

CatScience

4 часа назад

О регуляции сахара в крови

13 мин

4.8K

+10

20

3

rb_astana

23 часа назад

Первый месяц в Bug Bounty: итоги, цифры и выученные уроки

Простой

4 мин

8.4K

Ретроспектива

+8

24

9

Как мы разгрузили PostgreSQL и ускорили BI с помощью S3 и managed-сервисов

Промо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ

Исследуем, как айтишники относятся к ИИ в работе

ИТ-бренд + внешние коммуникации = удержание

Внедрил ML на заводе — опиши в новом сезоне Heavy Digital

СРЕДНЯЯ ЗАРПЛАТА В IT

223 415 ₽/мес.

— средняя зарплата во всех IT-специализациях по данным из 13 631 анкеты, за 1-ое пол. 2026 года. Проверьте «в рынке» ли ваша зарплата или нет!

https://habr.com/ru/articles/266831/

25/30

ЧИТАЮТ СЕЙЧАС

Прозрачный прокси для всей домашней сети на базе Xray: настройка за один вечер 20K  38 

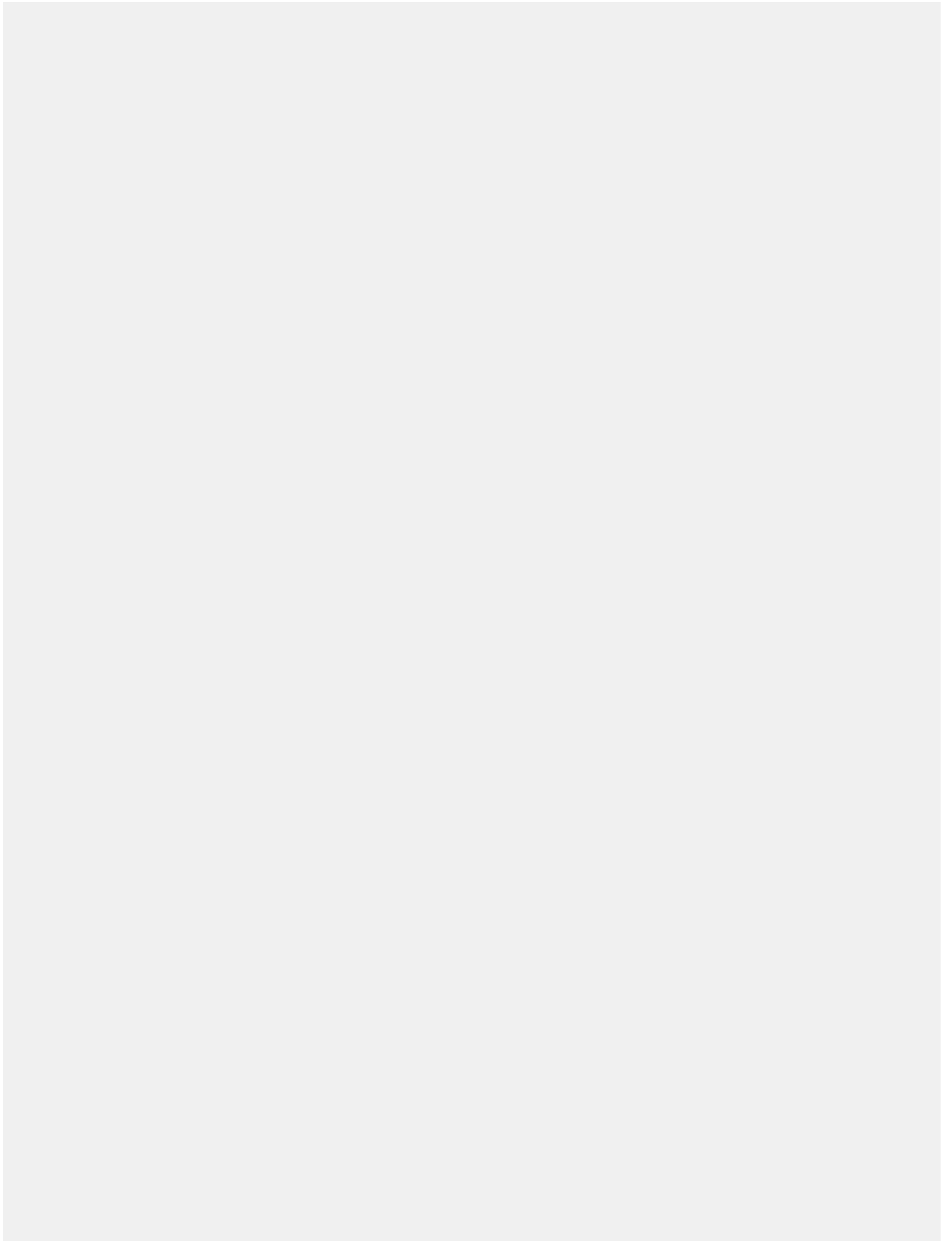
HeadHunter виноват в сломанном найме 26K  155 

Мы построили мир, который больше не понимаем или почему NASA не может скопировать свой же двигатель 237K  1173 

Новое поколение впервые за всю историю наблюдений реально оказалось глупее предыдущего 61K  246 

Удаленка работникам не выгодна 7.3K  97 

Как мы разгрузили PostgreSQL и ускорили BI с помощью S3 и managed-сервисов[Промо](#)

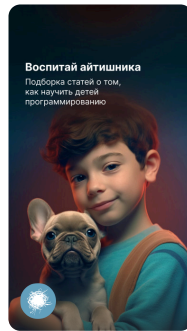


ИСТОРИИ



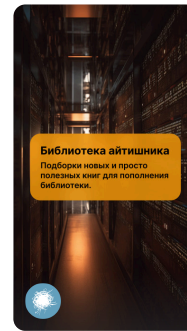
Что почитать?
Недельный топ-7 годноты
из блогов компаний

Годнота из блогов компаний



Воспитай айтишника
Подборка статей о том,
как научить детей
программированию

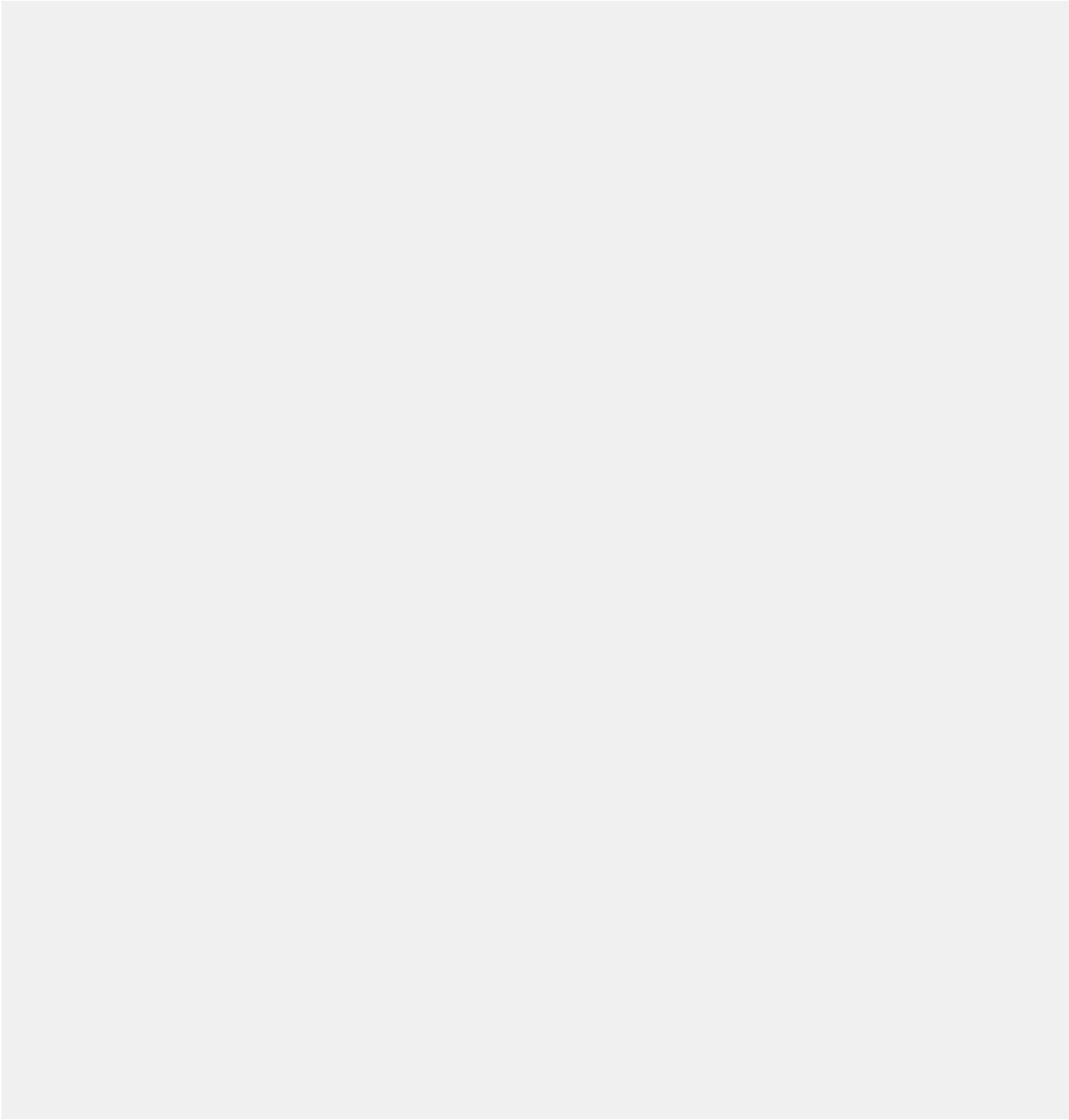
Как учить детей программированию



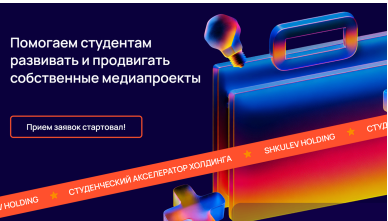
Библиотека айтишника
Подборка новых и просто
полезных книг для пополнения
библиотеки.

Почитаем, полюбопытствуем





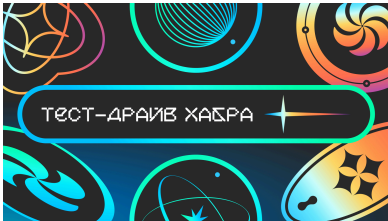
БЛИЖАЙШИЕ СОБЫТИЯ



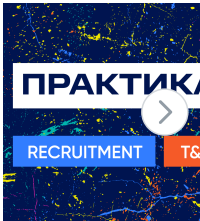
23 января – 1 марта



10 февраля



25 февраля



5 марта

Ваш аккаунт

Войти

Регистрация

Разделы

Статьи

Новости

Хабы

Компании

Авторы

Песочница

Информация

Устройство сайта

Для авторов

Для компаний

Документы

Соглашение

Конфиденциальность

Услуги

Корпоративный блог

Медийная реклама

Нативные проекты

Образовательные программы

Стартапам

VK

Telegram

YouTube

Apple

Настройка языка

Техническая поддержка

© 2006–2026, Habr